



**WYŻSZA SZKOŁA
INFORMATYKI i ZARZĄDZANIA**
z siedzibą w Rzeszowie

KOLEGIUM INFORMATYKI STOSOWANEJ

Kierunek: INFORMATYKA

Adrian Augustyn
Nr albumu studenta: 72886

"Programowanie Obiektowe – System Zarządzania Biblioteką"

Prowadzący: mgr inż. Ewa Żesławska

Projekt

Rzeszów 2025/2026

Spis treści

Wstęp	4
1 Opis struktury i technologii projektu	5
1.1 Diagram klas i opis struktury Poniższy diagram przedstawia strukturę obiektową systemu.	5
1.2 Opis techniczny i narzędzia Aplikacja została zbudowana przy użyciu następujących technologii	5
1.3 Minimalne wymagania sprzętowe Program jest lekką aplikacją konsolową, więc wymagania są minimalne	6
1.4 Zarządzanie danymi i bazą danych (BD)	6
1.4.1 Instrukcja konfiguracji i uruchomienia	6
2 Harmonogram realizacji projektu	7
2.1 Wizualizacja harmonogramu (Diagram Ganta)	7
2.2 Opis etapów realizacji	7
3 Repozytorium i system kontroli wersji	8
3.1 Link do kodu źródłowego	8
4 Prezentacja warstwy użytkowej projektu	9
4.1 Interfejs menu głównego	9
4.2 Wyświetlanie zasobów bibliotecznych	9
4.3 Operacja dodawania nowych pozycji (CRUD)	9
5 Podsumowanie	11
5.1 Zestawienie zrealizowanych prac	11
5.2 Planowane dalsze prace rozwojowe	11
Bibliografia	12
Spis rysunków	13
Spis tablic	14
Streszczenie	15

Wstęp

Głównym założeniem projektu było stworzenie funkcjonalnej aplikacji do zarządzania zbiorami bibliotecznymi, która zastąpi tradycyjne, papierowe metody ewidencji nowoczesną bazą danych SQL Server. System został zaprojektowany w języku C# na platformie .NET 8.0, co pozwala na sprawne przeglądanie, dodawanie oraz modyfikowanie zasobów biblioteki.

Celami projektu było nie tylko stworzenie narzędzia użytkowego, ale również praktyczne zastosowanie zasad programowania obiektowego. Dzięki wykorzystaniu hierarchii klas i mechanizmu dziedziczenia, zasoby takie jak książki, ebooki czy czasopisma zostały uporządkowane w logiczną strukturę.

Wymagania funkcjonalne i niefunkcjonalne

W ramach wymagań funkcjonalnych aplikacja realizuje podstawowe operacje na danych, umożliwiając użytkownikowi wyświetlanie pełnej listy zasobów, dodawanie nowych pozycji oraz usuwanie tych niepotrzebnych bezpośrednio z poziomu konsoli.

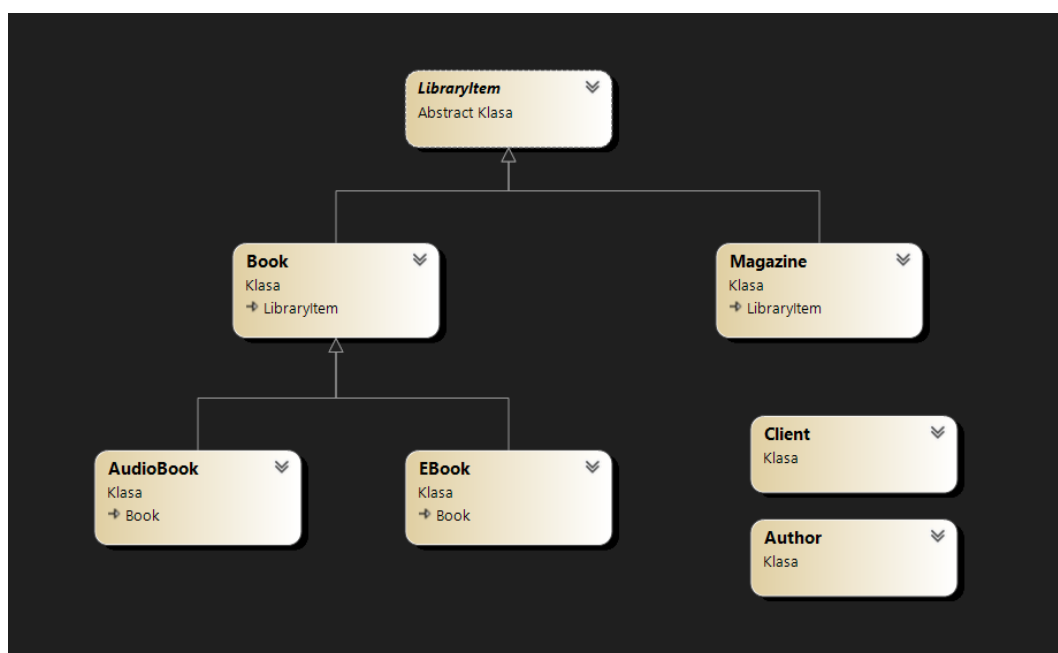
Pod względem technicznym (wymagania niefunkcjonalne) system zapewnia trwałość informacji poprzez zapis w relacyjnej bazie danych, co zapobiega ich utracie po zamknięciu programu. Program został zabezpieczony przed atakami typu SQL Injection oraz posiada mechanizmy obsługi wyjątków, które gwarantują stabilność pracy nawet w przypadku wystąpienia błędów. Poniższa dokumentacja zawiera szczegółowy opis techniczny, diagramy klas oraz instrukcję obsługi systemu.

Rozdział 1

Opis struktury i technologii projektu

W tej sekcji przedstawiono architekturę aplikacji oraz szczegóły techniczne dotyczące wykorzystanych narzędzi i sposobu zarządzania danymi.

1.1 Diagram klas i opis struktury Poniższy diagram przedstawia strukturę obiektową systemu.



Rysunek 1.1: Diagram klas UML przedstawiający strukturę dziedziczenia i relacje w systemie.

Struktura opiera się na klasie abstrakcyjnej `LibraryItem`, która definiuje wspólne cechy dla wszystkich obiektów (np. ID i Tytuł). Klasy `Book` oraz `Magazine` dziedziczą po niej te właściwości, a `Book` stanowi dodatkowo bazę dla klas `EBook` i `AudioBook`. Klasy `Author` i `Client` funkcjonują niezależnie, umożliwiając przechowywanie informacji o twórcach i użytkownikach biblioteki.

1.2 Opis techniczny i narzędzia Aplikacja została zbudowana przy użyciu następujących technologii

- **Język programowania:** C# w wersji 12.
- **Platforma:** .NET 8.0 (Console Application).

- **Środowisko:** Visual Studio 2022.
- **Baza danych:** Microsoft SQL Server (LocalDB / Express).

1.3 Minimalne wymagania sprzętowe Program jest lekką aplikacją konsolową, więc wymagania są minimalne

- **System operacyjny:** Windows 10/11.
- **Procesor:** Dowolny dwurdzeniowy (min. 1 GHz).
- **Pamięć RAM:** 512 MB wolnej pamięci (2 GB zalecane dla komfortu pracy z SQL Server).
- **Dysk:** ok. 50 MB na aplikację oraz miejsce na bazę danych.

1.4 Zarządzanie danymi i bazą danych (BD)

Zarządzanie informacjami odbywa się poprzez warstwę danych zintegrowaną z SQL Server za pomocą technologii ADO.NET. System realizuje operacje CRUD (dodawanie, odczyt i usuwanie rekordów). Komunikacja z bazą danych została zabezpieczona poprzez zapytania parametryzowane, co eliminuje błędy składniowe i chroni przed atakami typu SQL Injection. Dostęp do danych jest zamykany automatycznie po każdej operacji dzięki zastosowaniu bloków using w kodzie C#.

1.4.1 Instrukcja konfiguracji i uruchomienia

Ze względu na lokalny charakter bazy danych, uruchomienie projektu na innym komputerze wymaga edycji parametrów połączenia:

- **Krok 1:** Należy wykonać dołączony skrypt .sql w programie SSMS, co utworzy bazę o nazwie `library_72886`.
- **Krok 2:** W pliku `Program.cs` należy zaktualizować linię inicjalizacji serwera:

```
DatabaseService ds = new DatabaseService(  
    @"LENOVOAVG\SQLEXPRESS", "library_72886");
```

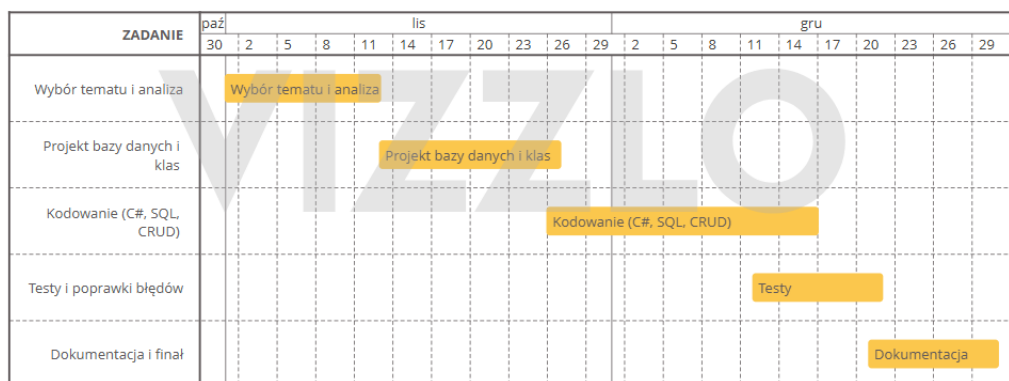
W miejsce `LENOVOAVG\SQLEXPRESS` należy wpisać nazwę własnej instancji serwera. Symbol `@` jest niezbędny do poprawnej obsługi znaku backslash.

Rozdział 2

Harmonogram realizacji projektu

Poniższa sekcja przedstawia szczegółowy plan prac nad systemem bibliotecznym, od momentu deklaracji tematu do finalizacji dokumentacji technicznej.

2.1 Wizualizacja harmonogramu (Diagram Ganta)



Rysunek 2.1: Diagram Ganta - harmonogram prac nad projektem

2.2 Opis etapów realizacji

Proces tworzenia aplikacji został podzielony na trzy kluczowe fazy:

- **Faza 'Wybór tematu i analiza' (1.11 – 12.11):** Określenie celu projektu oraz szczegółowa analiza wymagań funkcjonalnych.
- **Faza 'Projekt bazy danych i klas' (13.11 – 26.11):** Opracowanie architektury systemu. Powstał kompletny schemat bazy danych SQL oraz diagram klas.
- **Faza 'Kodowanie (C#, SQL, CRUD)' (26.11 – 16.12):** Intensywne prace implementacyjne w języku C# i SQL.
- **Faza 'Testy i poprawki błędów' (12.12 – 21.12):** Weryfikacja stabilności aplikacji i poprawności działania wszystkich funkcji.
- **Faza 'Dokumentacja i finał' (21.12 – 30.12):** Złożenie wszystkich elementów projektu w całość. Przygotowanie ostatecznej dokumentacji technicznej w systemie $\text{\LaTeX}$.

Rozdział 3

Repozytorium i system kontroli wersji

Po zakończeniu prac nad aplikacją i przeprowadzeniu testów, kod źródłowy został umieszczony w systemie kontroli wersji Git. Projekt jest hostowany w serwisie GitHub, co pozwala na łatwy dostęp do struktury plików oraz wgląd w implementację logiki biznesowej.

3.1 Link do kodu źródłowego

Kompletny projekt, zawierający wszystkie klasy C# oraz skrypty niezbędne do odtworzenia bazy danych SQL, znajduje się pod adresem: <https://github.com/mcadok/LibraryCRUD-CSharp>

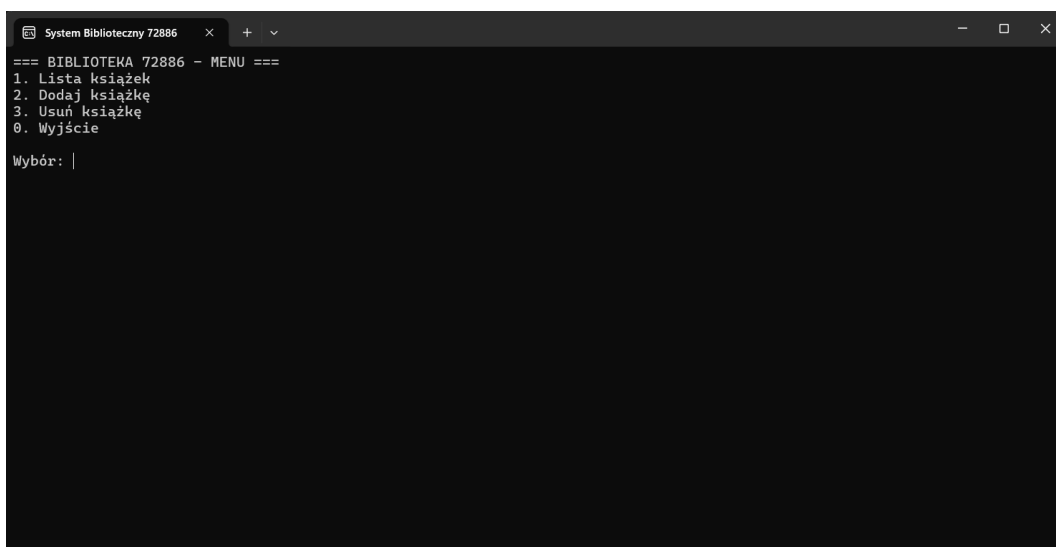
Rozdział 4

Prezentacja warstwy użytkowej projektu

Warstwa użytkowa aplikacji została zrealizowana w formie interfejsu konsolowego (CLI), co zapewnia szybkość działania i przejrzystość obsługi. Poniżej przedstawiono główne widoki programu oraz opisano proces interakcji z użytkownikiem.

4.1 Interfejs menu głównego

Po uruchomieniu systemu użytkownikowi prezentowane jest menu wyboru, które stanowi punkt wyjścia do wszystkich funkcjonalności aplikacji. Nawigacja odbywa się poprzez wpisanie numeru odpowiadającego wybranej akcji.



```
System Biblioteczny 72886 x + v
=== BIBLIOTEKA 72886 - MENU ===
1. Lista książek
2. Dodaj książkę
3. Usun książkę
0. Wyjście
Wybór: |
```

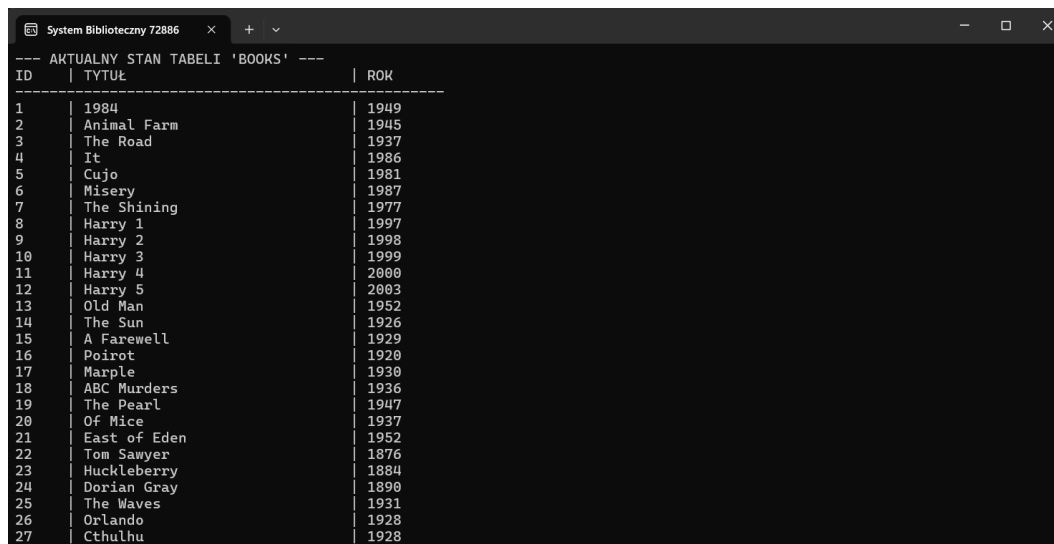
Rysunek 4.1: Główne menu aplikacji z dostępnymi opcjami zarządzania zasobami.

4.2 Wyświetlanie zasobów bibliotecznych

Po wybraniu opcji przeglądania, aplikacja łączy się z bazą danych SQL Server i pobiera aktualną listę rekordów. Dane są prezentowane w formie czytelnego zestawienia, zawierającego unikalny identyfikator ID, tytuł, autora oraz typ zasobu.

4.3 Operacja dodawania nowych pozycji (CRUD)

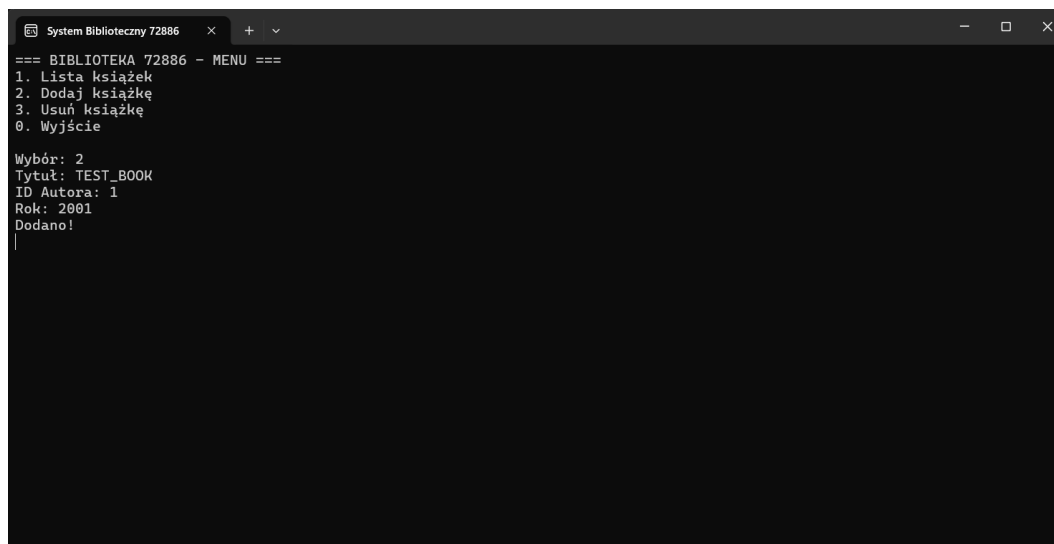
System umożliwia dynamiczne wprowadzanie nowych danych. Użytkownik jest proszony o podanie niezbędnych informacji, które są następnie walidowane i przesyłane do bazy danych za pomocą



ID	TYTUŁ	ROK
1	1984	1949
2	Animal Farm	1945
3	The Road	1937
4	It	1986
5	Cujo	1981
6	Misery	1987
7	The Shining	1977
8	Harry 1	1997
9	Harry 2	1998
10	Harry 3	1999
11	Harry 4	2000
12	Harry 5	2003
13	Old Man	1952
14	The Sun	1926
15	A Farewell	1929
16	Poirot	1920
17	Marple	1930
18	ABC Murders	1936
19	The Pearl	1947
20	Of Mice	1937
21	East of Eden	1952
22	Tom Sawyer	1876
23	Huckleberry	1884
24	Dorian Gray	1890
25	The Waves	1931
26	Orlando	1928
27	Cthulhu	1928

Rysunek 4.2: Widok listy książek pobranej bezpośrednio z relacyjnej bazy danych.

zapytań parametryzowanych. Program informuje o poprawnym zakończeniu operacji stosownym komunikatem.



```

=== BIBLIOTEKA 72886 - MENU ===
1. Lista książek
2. Dodaj książkę
3. Usuń książkę
0. Wyjście

Wybór: 2
Tytuł: TEST_BOOK
ID Autora: 1
Rok: 2001
Dodano!

```

Rysunek 4.3: Proces wprowadzania nowej pozycji do systemu i potwierdzenie zapisu w bazie.

Rozdział 5

Podsumowanie

Projekt systemu do zarządzania biblioteką został pomyślnie ukończony i dostarczony w wyznaczonym terminie. Wszystkie założenia postawione na początku pracy zostały zrealizowane, a aplikacja stanowi w pełni funkcjonalne narzędzie do ewidencji zbiorów bibliotecznych.

5.1 Zestawienie zrealizowanych prac

ramach projektu udało się osiągnąć następujące cele techniczne i użytkowe:

- **Integracja z bazą danych:** Stworzono stabilne połączenie z MS SQL Server, co zapewnia trwałość danych niezależnie od działania programu.
- **Implementacja CRUD:** Użytkownik ma możliwość pełnego zarządzania rekordami, w tym ich przeglądania, dodawania oraz usuwania z poziomu konsoli.
- **Wykorzystanie OOP:** Zastosowano zaawansowane mechanizmy programowania obiektowego, w szczególności dziedziczenie klas, co pozwoliło na logiczne uporządkowanie struktury zasobów.
- **Bezpieczeństwo i stabilność:** Zaimplementowano obsługę wyjątków oraz parametryzację zapytań SQL, chroniącą system przed błędami i atakami typu SQL Injection.

5.2 Planowane dalsze prace rozwojowe

Mimo że obecna wersja aplikacji jest kompletna pod kątem wymagań, w przyszłości projekt mógłby zostać rozbudowany o następujące elementy:

- **Interfejs graficzny (GUI):** Zastąpienie menu konsolowego oknami w technologii WPF lub WinForms, co zwiększyłoby komfort pracy użytkownika.
- **System logowania:** Wprowadzenie modułu autoryzacji dla bibliotekarzy, z podziałem na role i uprawnienia.
- **Integracja z API:** Możliwość automatycznego pobierania danych o książkach (np. opisu lub okładki) z zewnętrznych serwisów po wpisaniu numeru ISBN.
- **Rozbudowane raportowanie:** Eksport list książek i statystyk wypożyczeń do plików zewnętrznych (np. PDF lub Excel).

Bibliografia

- [1] **Dokumentacja Microsoft .NET** – oficjalne zasoby techniczne dotyczące platformy .NET 8.0 oraz języka C#.
<https://learn.microsoft.com/dotnet/>
- [2] **Dokumentacja Microsoft SQL Server** – materiały dotyczące projektowania baz danych oraz składni języka T-SQL.
<https://learn.microsoft.com/sql/sql-server/>
- [3] **Wzorce projektowe i zasady SOLID** – materiały dydaktyczne dotyczące programowania obiektowego i dziedziczenia klas.

Spis rysunków

1.1	Diagram klas UML przedstawiający strukturę dziedziczenia i relacje w systemie.	5
2.1	Diagram Ganta - harmonogram prac nad projektem	7
4.1	Główne menu aplikacji z dostępnymi opcjami zarządzania zasobami.	9
4.2	Widok listy książek pobranej bezpośrednio z relacyjnej bazy danych.	10
4.3	Proces wprowadzania nowej pozycji do systemu i potwierdzenie zapisu w bazie.	10

Spis tabel

Streszczenie projektu
Programowanie Obiektowe – System Zarządzania Biblioteką

Autor: Adrian Augustyn
Promotor: mgr inż. Ewa Żesławska
Słowa kluczowe: System zarządzania biblioteką w C# z operacjami CRUD

Projekt – Programowanie Obiektowe – System Zarządzania Biblioteką to aplikacja napisana w języku C#, realizująca podstawowe operacje CRUD (dodawanie, przeglądanie, edytowanie i usuwanie danych) dotyczące książek, czytelników oraz wypożyczeń. Wszystkie dane są przechowywane w plikach tekstowych, a struktura programu opiera się na klasach z hierarchią dziedziczenia (np. Osoba → Czytelnik/Bibliotekarz). Aplikacja ma na celu ułatwienie zarządzania biblioteką w prosty i przejrzysty sposób.

The University of Information Technology and Management in Rzeszow
Faculty of Applied Information Technology

Thesis Summary
Object-Oriented Programming – Library Management System

Author: Adrian Augustyn
Supervisor: mgr inż. Ewa Żesławska
Key words: Library Management System in C# with CRUD Operations

Project – Object-Oriented Programming – Library Management System is an application written in C#, implementing basic CRUD operations (adding, viewing, editing, and deleting data) for books, readers, and loans. All data is stored in text files, and the program structure is based on classes organized in a hierarchy (e.g., Person → Reader/Librarian). The application is designed to simplify library management in a clear and user-friendly way.